Integración de servicios Firebase en AstroCrate

Nombre: Alexander Valdés

Sección:

Fecha: 22/10/2026

Aplicación seleccionada

Para esta segunda evaluación se seleccionó la aplicación AstroCrate (Sistema de gestión de empaquetaría y envíos) siendo desarrollada por mi Alexander Gabriel Valdés Ibarra. Sobre esta versión base se procedió a migrar la arquitectura de datos para integrar el servicio de Google Firebase.

Descripción de las entidades y propósitos

Entidad Cliente: Representa a las personas o empresas que contratan el servicio de empaquetaría y envíos

- Campos propios (5): nombre (String), rut (String), direccion (String), telefono (String), email (String) + id (Document ID en Firestore).

Entidad Paquete: Representa las características físicas del paquete o carga.

- Campos propios (5): tipo (String: Sobre, Caja Chica, Caja Grande, Carga Irregular), contenido (String), peso (Double en Kg), esFragil (Boolean), valorDeclarado (Double en $) + id (Document ID en Firestore).

Entidad Envió: Asocia un cliente con un paquete para generar un guía de despacho con su estado de seguimiento.

- Campos propios (5): codigoSeguimiento (String: ej. AC-81303), clienteNombre (String), paqueteTipo (String), fechaEnvio (String), estado (String), precio (Double) + id (Document ID en Firestore).

Evolución del almacenamiento

Antes: Los datos se almacenaban temporalmente por memoria RAM mediante objetos “mutableListOf” dentro del singleton “DatosGlobales”. Al cerrar o reiniciar la aplicación, todos los registros ingresados o editados se perdían permanentemente.

Ahora: Se reemplazó el almacenamiento de RAM por Cloud Firestore. Todas las operaciones de creación, consulta, actualización y borrado se realizan de forma asíncrona directamente en la nube de Firebase. Al cerrar o forzar la detención de la app, la información persiste intacta en la base de datos de Firebase.

Estructura de colecciones en Cloud Firestore

Colección Clientes:


```
{
  "id": "auto_generated_id_123",
  "nombre": "Ignacio Vera Silva",
  "rut": "12.345.678-9",
  "direccion": "Av. Las Condes 1234",
  "telefono": "+56912345678",
  "email": "ignacio.vera@correo.com"
}
```


Colección Paquetes:


```
{
  "id": "auto_generated_id_456",
  "tipo": "Caja Chica",
  "contenido": "Telefono Motorola G-15",
  "peso": 2.5,
  "esFragil": true,
  "valorDeclarado": 150000.0
}
```


Colección Envios:


```
{
  "id": "auto_generated_id_789",
  "codigoSeguimiento": "AC-81303",
  "clienteNombre": "Ignacio Vera Silva",
  "paqueteTipo": "Caja Chica - Telefono Motorola G-15",
  "fechaEnvio": "30/10/2026",
  "estado": "En sucursal",
  "precio": 155000.0
}
```

Reglas de seguridad Cloud Firestore

Se configuraron reglas de acceso que exigen autenticación activa previa a cualquier operación de lectura o escritura, protegiendo las colecciones de accesos no autorizados:

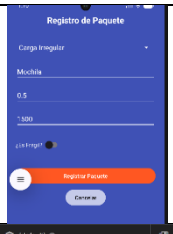
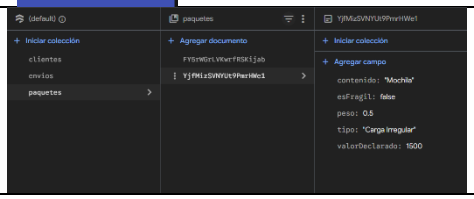
```
rules_version = '2';

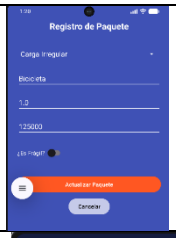
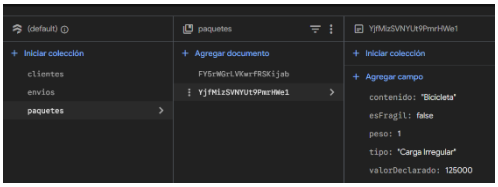
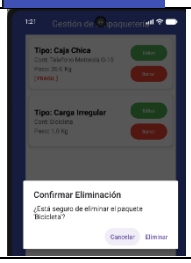
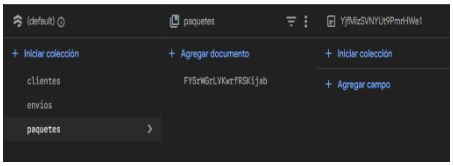

service cloud.firestore {
  match /databases/{database}/documents {


    // Regla de seguridad: Permite lectura y escritura únicamente a usuarios autenticados
    match /{document=**} {
      allow read, write: if request.auth != null;
    }
  }
}
```

- **Explicación:** La condición request.auth != null verifica que la petición provenga de una sesión autenticada con Firebase Authentication. Se evita el acceso público sin restricciones (allow read, write: if true), asegurando la integridad de los datos.

Evidencias del CRUD en Cloud Firestore

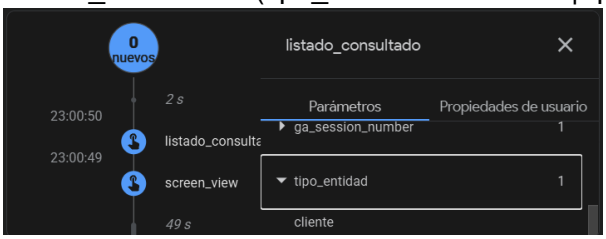
Operación	Descripción	Evidencia
Crear	Formulario de creación guardando en Firestore	
Consultar listado	Listado en RecyclerView leyendo datos desde la nube	

Actualizar	Carga de datos existentes y guardado de cambios	 
Eliminar	Confirmación mediante AlertDialog antes de eliminar	 

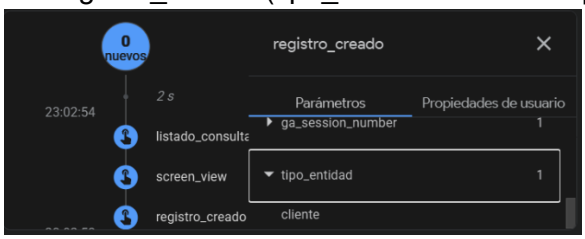
Eventos y Parámetros de Firebase Analytics

Se integraron 4 eventos específicos en cada una de las entidades, incluyendo el parámetro identificador tipo_entidad:

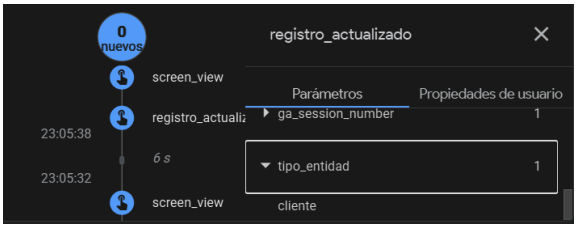
1. listado_consultado (tipo_entidad = "cliente" | "paquete" | "envio")



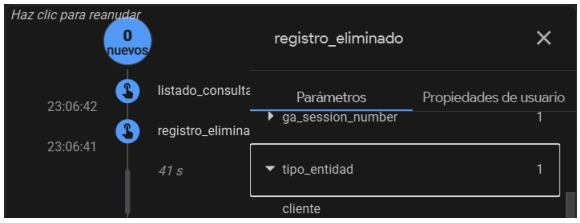
2. registro_creado (tipo_entidad = "cliente" | "paquete" | "envio")



3. registro_actualizado (tipo_entidad = "cliente" | "paquete" | "envio")

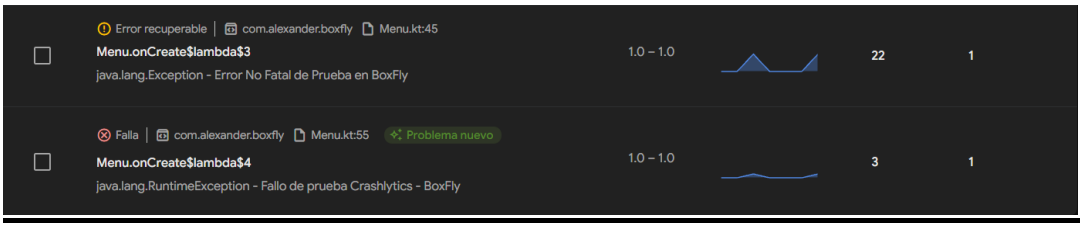


4. registro_eliminado (tipo_entidad = "cliente" | "paquete" | "envio")



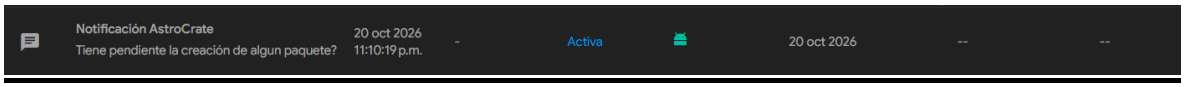
Evidencias de Firebase Crashlytics

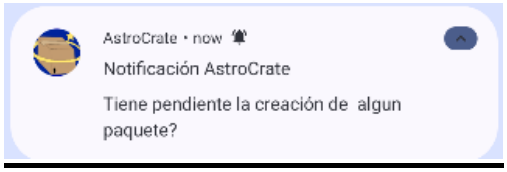
Error Recuperable es el “Error no fatal” mientras que la Falla con 3 eventos es el Crasheo generado por el botón “Testear Crash”



Evidencias de Firebase Cloud Messaging

- Se desarrolló el servicio MyFirebaseMessagingService.kt para interceptar mensajes en primer y segundo plano.
- Se configuró el canal de notificaciones y solicitud de permisos para Android 13+ (POST_NOTIFICATIONS).





Distribución del Trabajo y Dificultades

Integrante	Tareas realizadas	Evidencia de participación
Alexander Valdés	• Configuración de Firebase Auth (Login y Registro). • Migración del almacenamiento de RAM a Cloud Firestore para Clientes, Paquetes y Envíos con sus 5 campos propios y CRUD completo. • Lógica de actualización en cascada en Firestore. • Integración de Firebase Analytics con los 4 eventos exigidos y parámetro tipo_entidad. • Configuración de Firebase Crashlytics (errores no fatales y crash intencional). • Integración de Firebase Cloud Messaging (FCM), permisos de notificación y servicio de notificaciones push desde la consola.	Desarrollo, prueba e integración completa del proyecto AstroCrate en Android Studio
	Desarrollado de manera individual	

Instrucciones para Ejecutar las Pruebas

1. Compilación e Instalación: Abrir el proyecto en Android Studio y ejecutar en un dispositivo/emulador con Android 8.0 o superior.

2. Inicio de Sesión / Registro:

- Registrar un nuevo usuario con correo y contraseña válidos (mínimo 6 caracteres).
- Iniciar sesión con las credenciales registradas.

3. Prueba de CRUD y Persistencia:

- Crear un Cliente, un Paquete y un Envío.
- Editar un campo y eliminar un registro confirmando en el diálogo emergente.
- Cerrar por completo la app y reabrirla para comprobar que los datos persisten desde Firestore.

4. Prueba de Crashlytics: En el Menú principal, presionar "Error No Fatal" o "Testear Crash".

5. Prueba de FCM: Se hace un mensaje desde “Messaging” colocando su titulo, descripción y hora de ejecución luego de eso esperar a que llegue.